

Proposal for Converting Concordia PSH ERP into a Multi-Branch System

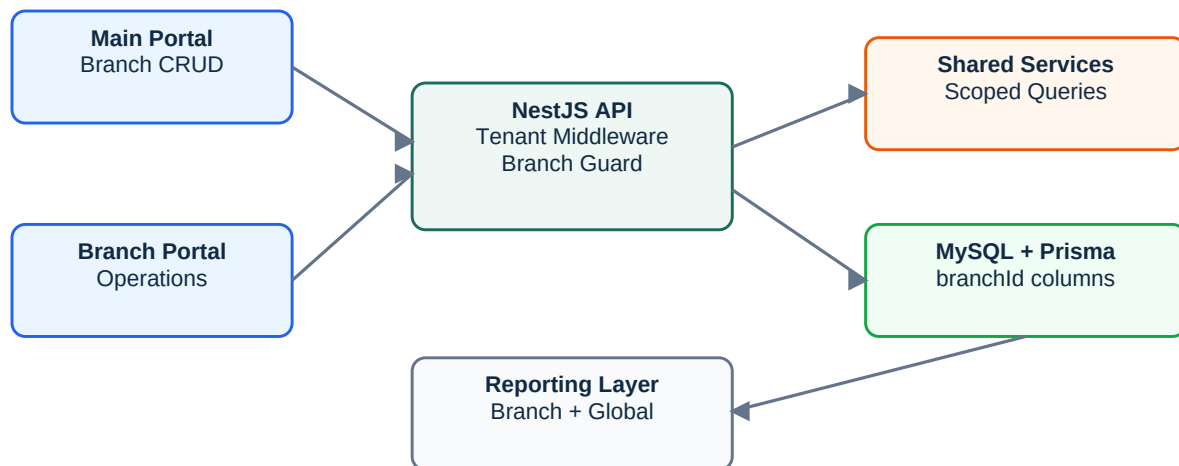
Main portal for branch CRUD, branch-scoped operations, consolidated reporting, and secure data isolation

Prepared for: Higher Authorities / Management Committee

Prepared by: Development Team

Date: 14 August 2026

Decision requested: Approval to design and implement a branch-aware ERP platform using the existing NestJS, React, Prisma, and MySQL foundation.



Target system keeps one codebase and one shared database, but every operational row is scoped by branch.

1. Executive Summary

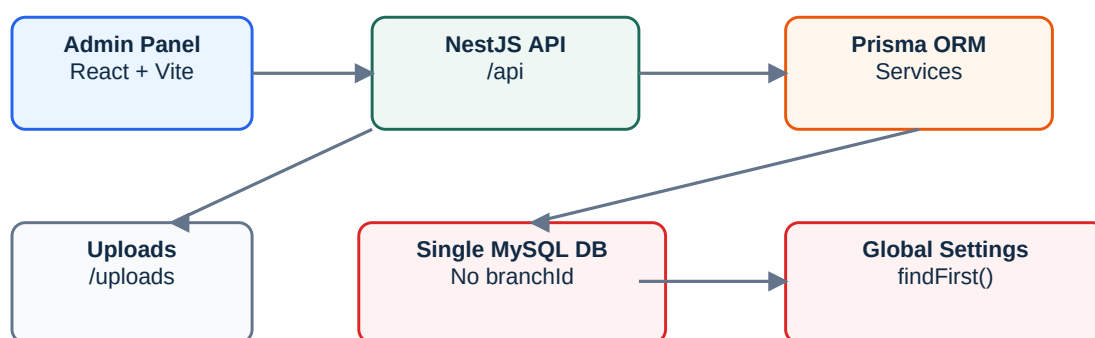
- The current ERP is a strong single-institute system covering admissions, academics, attendance, fee management, HR/payroll, finance, hostel, inventory, front office, examination, configuration, and dashboards.
- The proposed upgrade converts it into a multi-branch platform where head office manages N branches from a main portal, and each branch performs daily operations inside its own isolated workspace.
- Recommended approach: one shared application and one shared MySQL database, with strict `branchId` scoping on all branch-owned data.
- This approach reuses the current codebase, minimizes infrastructure cost, supports consolidated reporting, and avoids maintaining separate deployments for every branch.
- The main work is not only UI branch CRUD; it requires tenant context in auth, Prisma models, guards, service queries, reports, templates, migrations, and tests.

Business value

Value Area	Expected Outcome
Governance	Head office sees all branches, creates branches, manages branch admins, and can suspend/reactivate branches.
Operational control	Each branch manages its own students, fees, attendance, payroll, inventory, hostel, exams, and front-office data.
Data security	Users only access assigned branch data; cross-branch leakage is blocked by backend guard and query scoping.
Reporting	Management can compare branches and view consolidated finance, fee, enrollment, attendance, HR, hostel, and examination KPIs.
Scalability	N branches can be added without copying the whole application for each campus.

2. Current Codebase Analysis

Reviewed code shows a modular NestJS backend, React admin panel, Prisma schema, and MySQL database. The current system already has the operational depth needed for branches, but it is not yet branch-aware.



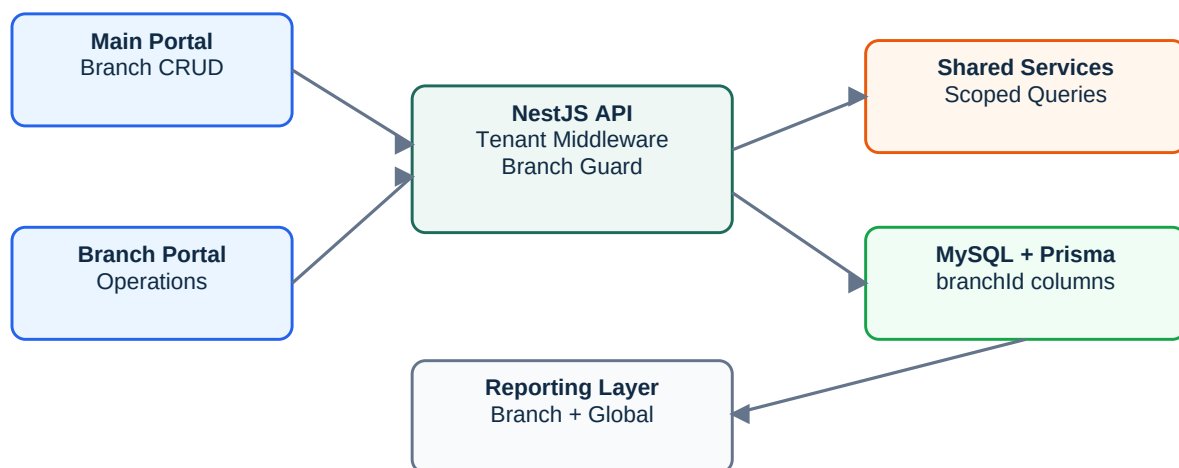
Current system serves one institute context. Branch isolation is not encoded in the database or auth context.

Layer	Current Finding	Multi-Branch Impact
Frontend	React + Vite admin panel with permission-aware navigation and hardcoded API base URL.	Add branch selector, main portal routes, branch-aware API client, branch dashboard context.
Backend	NestJS modules for auth, admin, academics, students, attendance, HR, front office, exams, config, hostel, inventory, fees, finance, dashboard.	Introduce `BranchModule`, tenant middleware, branch guard, branch-scoped services.

Layer	Current Finding	Multi-Branch Impact
Database	Prisma schema has 50+ models, but no global `branchId` on operational tables.	Add `Tenant` and `Branch`, then link branch-owned tables and update unique indexes.
Settings	`InstituteSettings.findFirst()` is used as singleton settings in fee/student/hostel/config flows.	Settings must become per branch, with optional head-office defaults.
Auth	JWT cookies identify user and role; permissions are JSON module/submodule flags.	JWT/session must include allowed branch IDs, active branch ID, and global-vs-branch role.
Reports	Dashboard and reports aggregate whole database.	Reports must support branch filter, branch comparison, and all-branch consolidated mode.

3. Proposed Multi-Branch Vision

- Main Portal: head office users create, update, deactivate, and monitor branches.
- Branch Portal: branch users work in selected branch context across existing modules.
- Multi-branch users: selected staff/admin can switch between permitted branches.
- Consolidated reports: head office can compare branches and drill down into a specific branch.
- Shared codebase: same backend/frontend deployment serves all branches.
- Strict isolation: all branch-owned reads/writes include backend-enforced branch scope.



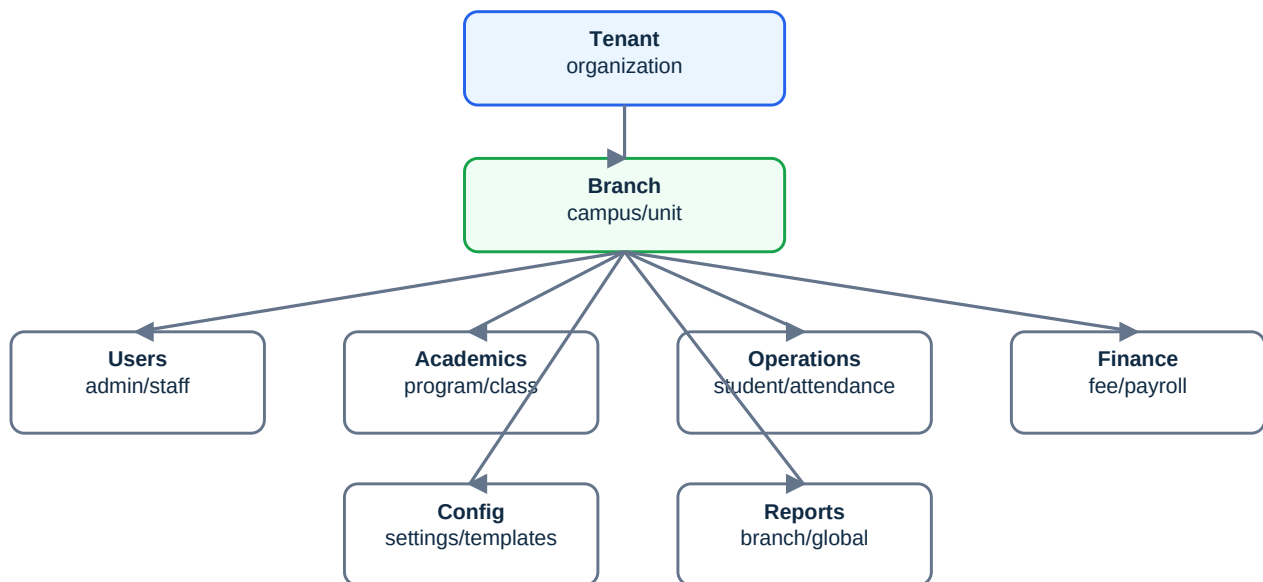
Target system keeps one codebase and one shared database, but every operational row is scoped by branch.

4. Recommended Architecture

Use shared-database multi-tenancy with branch row-level scoping. This fits the existing Prisma/MySQL architecture and keeps reporting simple.

Option	Description	Pros	Cons	Recommendation
A. Shared DB + branchId	One database, operational rows carry `branchId`.	Lowest cost, easy consolidated reporting, one deployment.	Requires careful query scoping and migration.	Recommended
B. Database per branch	Each branch has separate DB/schema.	Strong physical isolation.	Hard reporting, high maintenance, N migrations.	Not recommended now
C. App per branch	Separate frontend/backend per branch.	Quick for one-off branch.	Operational duplication and weak head-office control.	Reject

5. Target Data Model



Recommended model: Tenant owns N branches. Branch owns all operational data. Shared catalogs remain optional.

New core tables:

Table	Purpose	Important Fields
Tenant	Top-level organization/institute group.	`id`, `name`, `status`, `createdAt`, `updatedAt`
Branch	Branch/campus/unit managed from main portal.	`id`, `tenantId`, `name`, `code`, `city`, `address`, `phone`, `email`, `status`, `logo`, `settings`
UserBranchAccess	Maps admin/staff users to branches and permissions.	`id`, `userId`, `branchId`, `role`, `permissions`, `isDefault`
BranchSettings	Branch-specific institute and financial settings.	`branchId`, `instituteName`, `lateFeeRatePerDay`, `hostelLateFee`, `challanPrefix`, `templateDefaults`
BranchAuditLog	Audit trail for branch creation, updates, context switches, and sensitive actions.	`actorId`, `branchId`, `action`, `metadata`, `createdAt`

Branch-owned existing tables should receive `branchId`:

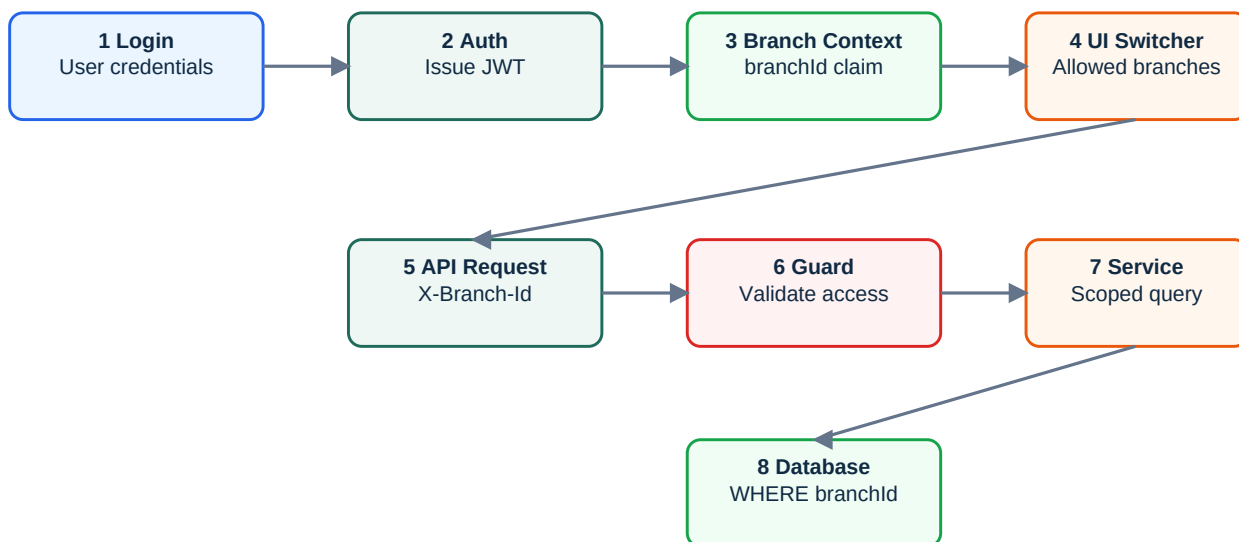
- Academic structure: Department, Program, Class, Section, AcademicSession, Subject, mappings, timetable.
- People: Student, StudentAcademicRecord, Staff, staff settings, admin branch access.
- Attendance: Attendance, AttendanceSkip, Leave, StaffAttendance, Holiday.
- Front office: Inquiry, Visitor, Complaint, ComplaintRemark, Contact.
- Examination: Exam, ExamSchedule, Marks, Result, Position, ReportCardTemplate.
- Finance and fees: FeeHead, FeeStructure, FeeInstallment, FeeChallanV2, payments, ExtraChallan, FinanceIncome, FinanceExpense, FinanceClosing.
- Hostel and inventory: HostelRegistration, Room, RoomAllocation, HostelExpense, HostelInventory, HostelChallan, SchoolInventory, InventoryExpense.
- Configuration/templates: InstituteSettings should become branch-scoped or replaced by BranchSettings; templates can be branch-specific with head-office defaults.

Unique keys must become branch-aware. Example: `Student.rollNumber` should become unique by `[branchId, rollNumber]`; department names by `[branchId, name]`; challan numbers by `[branchId, challanNumber]` unless a global prefix is required.

6. Authentication and Authorization Design

- Login returns user identity plus allowed branches.

- JWT access token includes stable user identity and allowed branch IDs; active branch can be sent in `X-Branch-Id` or stored in a signed active-branch cookie.
- Backend `BranchContextMiddleware` resolves branch context for every request.
- `BranchGuard` verifies the user is allowed to access the requested branch before controller/service logic runs.
- Head-office users can use `scope=all` on approved reporting endpoints only; CRUD endpoints require one active branch unless endpoint is main portal branch CRUD.
- Permissions remain module/submodule based but become branch-specific where needed.



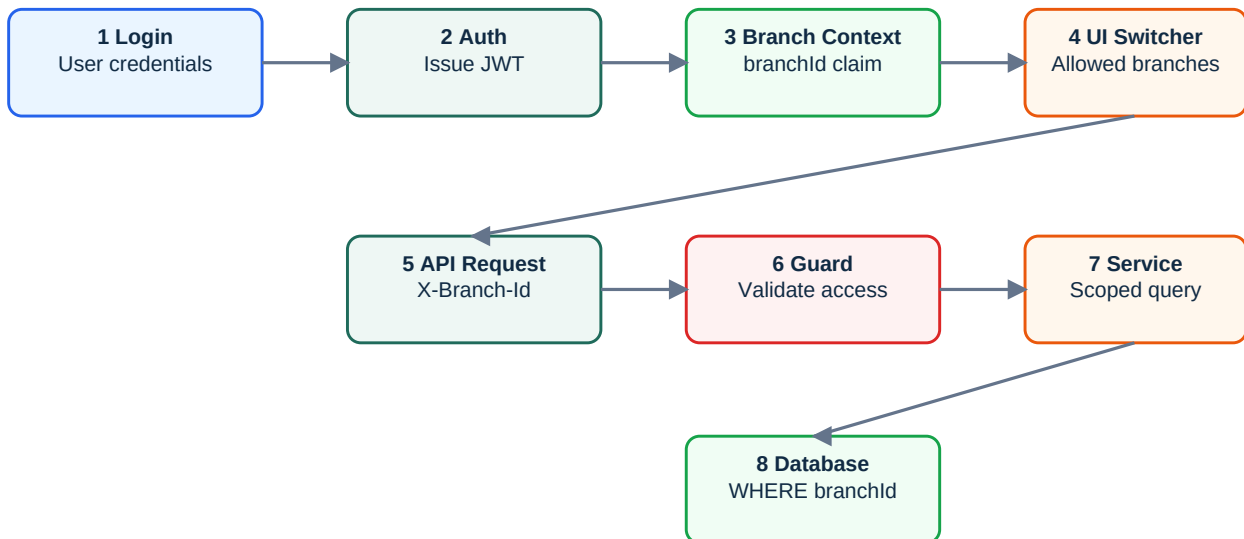
Each request carries a branch context; unauthorized branch access is rejected before service logic.

7. Main Portal Functional Scope

Module	Functions
Branch Management	Create branch, update profile, upload logo, activate/deactivate, assign branch code/prefix, configure address/contact.
Branch Admins	Create branch admin, assign one or many branches, set default branch, grant module permissions.
Branch Settings	Per-branch institute name, fee rates, challan prefix, hostel late fee, ID/report/challan templates.
Consolidated Dashboard	Branch count, total students, fee collection, expenses, payroll, attendance, hostel occupancy, exam performance.
Branch Comparison	Compare enrollment, revenue, arrears, staff count, expenses, attendance, pass rates.
Audit and Governance	Track branch CRUD, permission changes, finance approvals, data exports, and cross-branch report access.

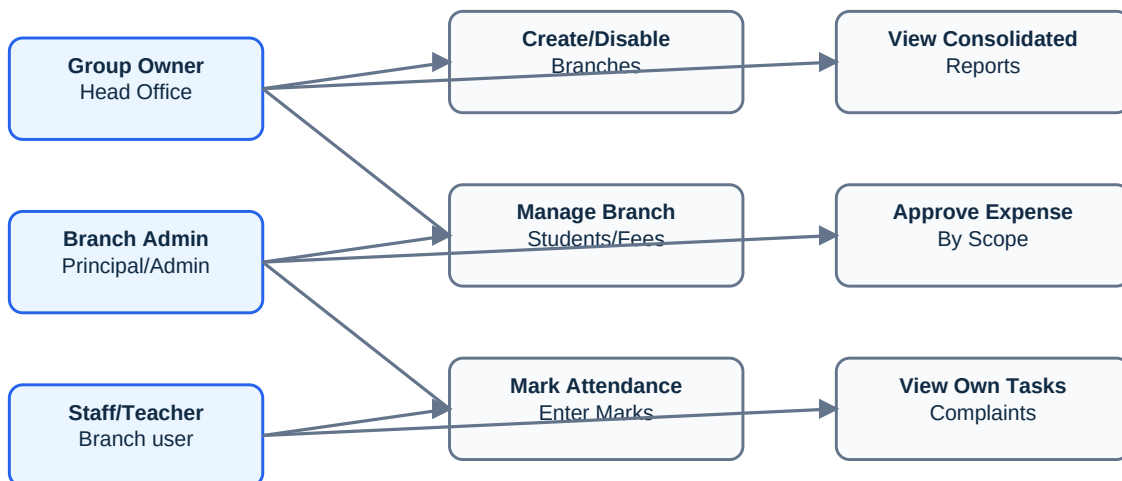
8. Branch Portal Data Flow

- User logs in and selects an active branch.
- Frontend stores active branch in app state and sends it with every API request.
- Backend validates branch access through guard/middleware.
- Service layer uses branch-aware Prisma wrapper or helper to add `branchId` to every query.
- Database returns only rows from active branch.
- Reports aggregate selected branch; head office can request all-branch aggregation from approved endpoints.



Each request carries a branch context; unauthorized branch access is rejected before service logic.

9. User Stories / Use Case View



User stories are branch-aware: head office sees all branches; branch users see assigned branches only.

Actor	User Story	Acceptance Criteria
Head Office Super Admin	As head office, I can create N branches from main portal.	Branch appears in list, has unique code, default settings, and status ACTIVE.
Head Office Super Admin	As head office, I can assign admins to one or many branches.	Assigned user can only open permitted branches and modules.
Branch Admin	As branch admin, I can manage students, fees, staff, attendance, and exams for my branch.	CRUD actions create records with active `branchId`; other branch data never appears.
Finance Manager	As finance manager, I can view branch-level and consolidated finance reports.	Branch filter works; all-branch mode available only to authorized users.
Teacher	As teacher, I can mark attendance only for assigned branch/classes.	Teacher class list and attendance rows are scoped to branch and assignment.
Management	As management, I can compare branch KPIs.	Dashboard shows side-by-side branch metrics and drill-down links.

10. API Changes

- Add `/api/branches` for create, list, update, deactivate/reactivate, get branch detail.
- Add `/api/branches/:id/settings` for branch settings.
- Add `/api/branches/:id/admins` or `/api/user-branch-access` for assignments.
- Add `X-Branch-Id` request header or equivalent branch context for all operational endpoints.
- Add `branchId` query support on head-office reporting endpoints.
- Keep existing endpoints stable where possible; enforce branch context internally to avoid frontend rewrite explosion.
- Return active branch and allowed branches in `/api/auth/user-who`.
- Reject CRUD request without branch context unless endpoint is explicitly global/main-portal.

Endpoint Group	Current	Proposed Change
Auth	`/auth/login`, `/auth/user-who`	Return allowed branches, default branch, active branch capability.
Main Portal	Not present as backend entity	New `BranchModule` with branch CRUD and settings.
Operational APIs	Existing routes like `/student/*`, `/fee/*`, `/hr/*`	Require branch context and add backend query scoping.
Reports	Aggregate whole DB	Add branch filter and authorized `all` scope.
Configuration	Global singleton settings	Branch settings plus optional head-office defaults.

11. Module-by-Module Impact

Module	Impact
Dashboard	Add branch filter, consolidated KPIs, branch comparison cards.
Configuration	Uncomment/replace branch UI with backend-driven branch CRUD; add branch settings and template defaults.
Admin/Auth	Branch assignment, branch-aware permissions, branch switch, global head-office role.
Academics	Branch-scoped sessions/programs/classes/sections/subjects/timetables.
Students	Branch-scoped admission, roll numbers, statuses, reports, academic records.
Attendance	Branch-scoped generation, marking, skips, leaves, holidays.
Fees	Branch challan prefixes, branch-scoped fee heads/structures/installments/payments, consolidated arrears.
HR/Payroll	Branch staff, payroll settings, staff attendance, leave policies, advance salary.
Finance	Branch expenses/income/closings; head-office approval matrix if required.
Hostel	Branch hostels/rooms/challans/inventory; optional multiple hostels per branch later.
Inventory	Branch stock, branch inventory expenses, transfer option in future phase.
Examination	Branch exams, marks, results, positions, consolidated performance.
Front Office	Branch inquiries, visitors, complaints, contacts.

12. Migration Strategy

- Create one default branch representing the current Concordia PSH branch.
- Add nullable `branchId` columns first, backfill all existing operational data to default branch, then make critical `branchId` columns required.
- Update unique constraints to include `branchId` before enabling more branches.
- Introduce branch guard and scoped Prisma helper behind feature flag.
- Convert singleton settings to branch settings while preserving existing values.

- Validate fee, payroll, attendance, and finance reports after backfill because they are high-risk financial/operational areas.
- Enable main portal branch CRUD only after isolation tests pass.

13. Delivery Roadmap

Phase	Duration	Key Deliverables
Phase 0 - Discovery	3-5 days	Finalize branch rules, data ownership matrix, acceptance criteria, report list.
Phase 1 - Foundation	1-2 weeks	Tenant/Branch schema, branch access table, migration/backfill, branch middleware/guard.
Phase 2 - Main Portal	1-2 weeks	Branch CRUD, branch settings, branch admin assignment, branch switcher.
Phase 3 - Core Modules	3-5 weeks	Branch scoping for academics, students, attendance, fees, HR, finance, dashboard.
Phase 4 - Secondary Modules	2-3 weeks	Hostel, inventory, front office, exams, templates, consolidated reports.
Phase 5 - QA/UAT	2 weeks	Isolation tests, migration tests, finance reconciliation, branch-user UAT.
Phase 6 - Rollout	1 week	Production migration, training, monitoring, handover.

14. Risks and Controls

Risk	Severity	Control
Cross-branch data leakage due to missed query filter.	High	Central branch-scoped Prisma helper, guard tests, code review checklist, automated isolation tests.
Financial report mismatch after branch migration.	High	Backfill audit, before/after totals, challan/payment reconciliation, finance sign-off.
Global singleton settings used in fee/hostel/student services.	High	Replace `findFirst()` settings calls with branch settings resolver.
Unique constraints break when multiple branches use same roll/challan numbers.	Medium	Convert to branch-aware unique indexes and branch prefixes.
Frontend pages forget branch context.	Medium	Central Axios interceptor adds `X-Branch-Id`; route guard blocks no-branch operations.
Scope creep around inter-branch transfers.	Medium	Make transfer workflows Phase 2.5 or later unless required immediately.

15. Clarifications Required Before Final Estimation

- Should one admin/staff user be allowed to work in multiple branches, or should every branch have separate user accounts?
- Should student roll numbers and challan numbers be unique globally or only within a branch with branch prefix?
- Should head office approve branch expenses, or should each branch approve its own finance records?
- Will hostel be one hostel per branch, or can a branch have multiple hostel buildings?
- Should academic sessions/programs/classes be shared across branches or configurable per branch?
- Should templates be managed centrally, per branch, or both with inheritance?
- Do higher authorities need branch comparison dashboards on day one, or only branch CRUD and isolation first?

16. Approval Request

Approval is requested to proceed with Phase 0 discovery and Phase 1 foundation work. The proposed path keeps the existing ERP investment, adds branch governance, protects branch data, and enables future institutional growth without deploying separate systems per branch.

Appendix A - Evidence From Current Codebase

- Backend modules imported in `AppModule`: Auth, Prisma, Admin, Department, Teacher, Academics, Student, Attendance, HR, Front Office, Examination, Configuration, Hostel, Inventory, Fee Management, Finance, Dashboard, Local File.
- Global API prefix is `/api`; frontend API base URL is currently `http://localhost:3003/api`.
- Prisma uses MySQL and defines 50+ operational models and enums.
- Existing branch UI code appears commented in Configuration page, but backend has no Branch model or Branch controller.
- Institute settings are singleton-like and read with `findFirst()` in fee, hostel, student, and configuration services.
- Navigation already contains all major ERP modules and submodules, so branch context can be added centrally rather than rebuilding navigation.